Redis高可用

怎么保证redis的高可用

- 主从, 集群, 哨兵.

主从

什么是主从, 为什么要有主从?

如果说你部署了单机的redis, 那这个redis挂了, 大量请求就会打到数据库. 怎么办?

- 可以部署多个redis, 做主从.
- 写数据只写入主节点, 然后从节点只能读.
- 然后通过主从复制, 把主节点的数据复制到从节点.

主从复制的过程

主从复制分为全量复制和增量复制过程.

- 全量复制就是说新启动了一个从节点, 从节点没有数据, 这时候就要复制主节点的全量数据给到从节点. 或者说有一个从节点网络断开了, 过了一段时间又连上主节点了, 这就要进行全量复制.
    - 全量复制就是主节点执行bgsave命令来生成RDB文件，然后把文件发送给从服务器。从服务器收到RDB文件后，会先清空当前的数据，然后加载RDB文件
    - 但是有个问题, 生成rdb文件过程中主节点也会有写命令进来, 这些命令怎么办?

    - 可以暂时放入缓冲区, 然后发送完rdb后, 再发送缓冲区中的命令, 这样redis从节点就收到了全部数据.
- 增量复制就是说, 主从在完成全量复制之后, 会和从节点维持一个tcp长连接. 后续的主节点收到的写命令, 就会基于长连接异步的发送给从节点.

当然了, 你可能会在网上看到很多说法, redis这个版本主从复制是怎么做的, 那个版本又是怎么做的, 哪些细节上又有什么差异. 不可否认, 这些文章很详细也很好. 但是大家不要去记这么详细的东西, 你今天记住了, 下周你就忘了, 面试也不需要给面试官这么详细的聊. 所以你只需要知道全量复制和增量复制怎么做就够了. 剩下的版本差异的那些细节没那么重要.

为什么全量复制要用rdb, 不用aof呢? aof不也是日志吗?

- 首先RDB是压缩后的二进制数据, 更小. aof是写命令, 更大.
- 而且rdb里面存的就是数据, 从节点接收rdb之后直接解析读入内存就行, 很快. aof是写命令, 从节点需要一条条执行命令, 很慢.

哨兵

为什么要有哨兵?

redis主从的情况下, 主节点挂了, 你需要手动地选个从节点, 然后转从为主. 太费劲了, 还容易出错.

有没有一种方式, 可以直接帮我选主, 然后转从为主呢?

- redis的哨兵就能监控所有redis节点的状态, 如果它发现主节点挂了, 就能自动帮你选主, 然后转从为主.

哨兵怎么发现主节点挂了呢?

- 首先, 哨兵会和其他节点维持一个心跳, 其实就是发送ping命令. 如果说某个节点在长时间没有回应, 那这个哨兵就会标记这个节点为主观下线.
- 为啥是主观下线呢? 因为, 哨兵收不到某个节点的响应, 有可能是哨兵自己有问题, 也可能只是网络不好, 所以不一定是这个节点挂了.
- 所以为了避免误判, 哨兵一般会部署3个节点以上的集群. 当一个哨兵判断主节点为主观下线后, 就会向其他哨兵发送命令, 其他哨兵根据自己监控的情况去投赞成票或者拒绝票.
- 如果说半数以上的哨兵都认为主节点挂了, 那主节点就会被认为是客观下线.

哨兵怎么做主从故障转移?

主节点被判定客观下线了, 哨兵怎么自动帮你选主呢? 怎么转从为主呢?

- 哨兵起码3个以上, 所以哨兵必须选出一个哨兵leader, 然后由哨兵leader去做主从故障转移.
- 哨兵leader的选举, 用了raft算法的leader选举思想.
    - raft算法是一个分布式一致性算法, 其中一个部分就是leader选举, 想要了解的可以网上搜一搜, 还是很复杂的.
- 选出的哨兵leader会从所有从节点里面, 挑选出一个从节点转从为主.

    - 具体规则可以看 `哨兵怎么选出新主节点?`
- 哨兵会向让其他从节点发送一条命令, 把新主节点的信息个告诉各个从节点, 从节点就会把复制日志的目标改为新主节点
- 将新主节点的IP地址和信息, 通过发布订阅机制通知给客户端.
    - 具体方式可以看 `哨兵怎么把新主节点的IP地址和信息, 通过给客户端呢?`
- 它还会继续监视挂了的旧主节点, 当这个旧主节点重新上线时, 把它设置为从节点

哨兵怎么选出新主节点?

- 首先排除已经下线的从节点, 然后从下面三大维度筛选
- slave优先级
    - 从节点是有优先级的, redis配置文件中replica-priority就是优先级.
    - 这个值越小, 优先级就越高.
- 复制进度(日志完整度)
    - 哨兵希望选择出数据最完整的, 也就是与旧主节点数据最接近的, 也就是复制进度最快的从节点作为新主节点.
    - 所以复制进度越高的, 就越容易成为新主节点
- 节点id
    - 每个节点都有一个节点id
    - 当有多个从节点的优先级和复制进度一样的话, 那节点id小的从节点就会成为新的主节点

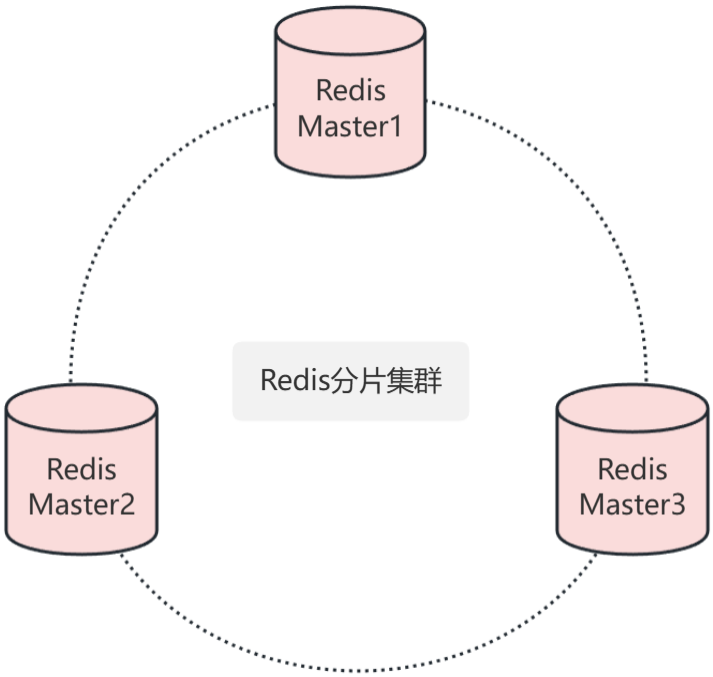
哨兵怎么把新主节点的信息, 通过给客户端呢?

- 客户端和哨兵建立连接后，客户端会订阅哨兵提供的频道.
- 主从切换完成后，哨兵就会向 `+switch-master` 频道发布新主节点的信息, ip端口等 (发布订阅机制)
- 客户端就可以收到信息后, 就会去和新主节点进行通信

集群

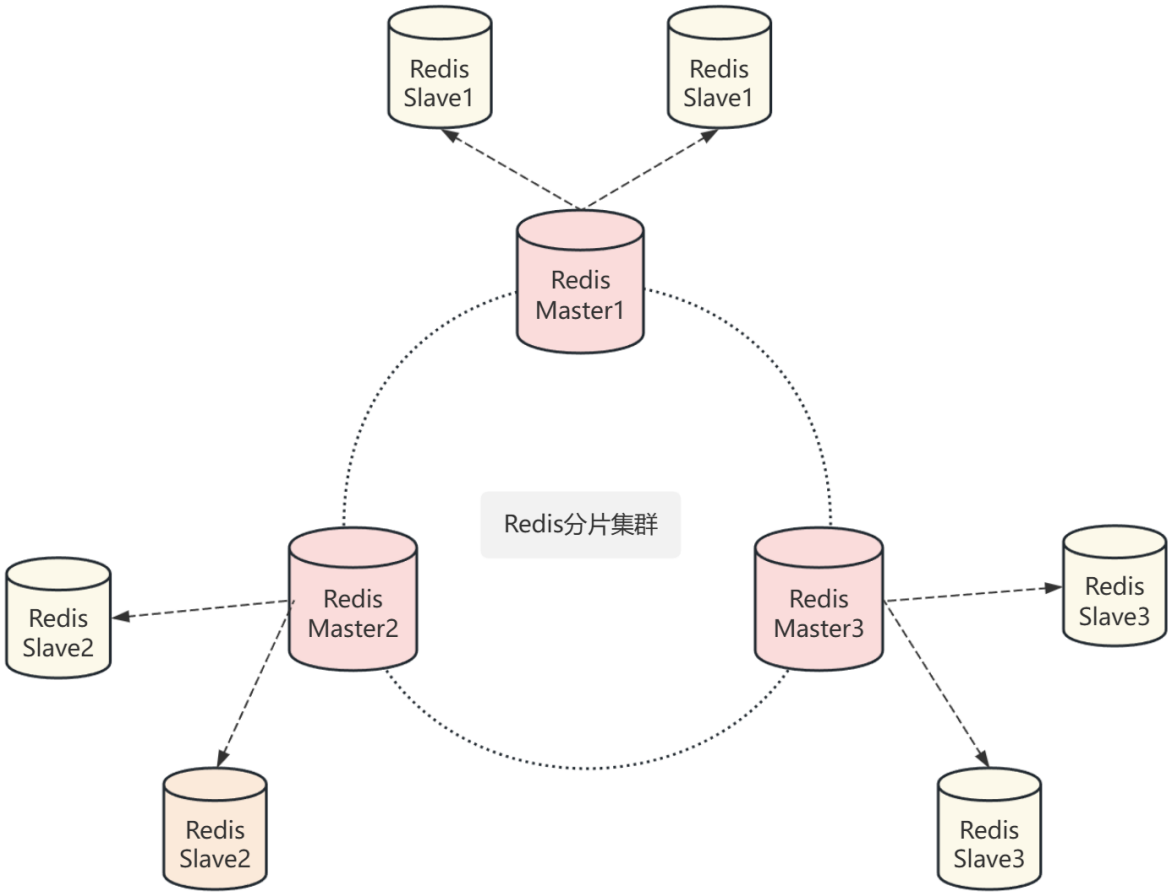
为什么需要集群?

- 先总结一下主从和哨兵:
  - 为了避免redis单点故障, 所以要做主从, 写操作只写入主库, 从库只负责读. 从库通过主从复制来同步主库数据.
  - 然后使用哨兵进行监控, 如果主节点故障, 哨兵就能自动选主, 然后转从为主, 最后通知从节点和客户端.
- 考虑两个问题
  - 因为redis主从存的都是全量的数据, 那一旦有大量的数据都要写入redis, 单机的内存可能就不够用了. 而且数据量大也会影响redis rdb持久化的性能.
  - 因为写操作只能在主节点上执行, 那主从的写操作就会有单机性能问题, 无法扩展
- 基于这两个问题, 就有了redis分片集群.
  - 把部署多个redis节点构成集群, 把大量数据分散存储在不同redis节点上, 这样每个redis节点都只保存一部分数据, 这就解决了缓存大数据量的问题.
  - 而且每个节点都可以用来写, 这就解决了写操作的单机瓶颈. 后续拓展, 直接新增节点就行.



那要是redis集群中某个节点挂了怎么办?

- 所以为了保证集群的高可用, 集群中每个节点都要单独做主从.



redis集群是怎么做分片的?

redis集群数据分片主要是用了16384个hash槽. 创建redis集群的时候, redis就会自动平均分配16384个槽到各个节点, 每个节点只负责一部分hash槽.

对redis做读写的时候, 一个key进来, 使用CRC16算法计算一个值, 然后对16384取余, 就能算出这个key应该被分配到哪个槽中, 然后再找这个槽对应的redis节点去读写就可以了.

- hash槽 = CRC16(key) mod 槽数量
- 当有16384个槽时: hash槽 = CRC16(key) mod 16384

那么问题又来了, redis集群扩容缩容, 那么hash槽就会迁移到其他节点. 那你访问的这个key对应的槽正在迁移怎么办, 这个时候redis集群还能正常提供服务吗?

- 当然是可以的.
- 集群在扩容缩容的时候, 部分槽会迁移.
- 所以key到了对应的节点之后, 有可能会发现, 这个槽不是当前节点负责的
- 这时候就会触发重定向, 告诉客户端, 目前这个槽是哪个节点负责的.
- 那客户端再重新请求对应节点.

各个节点怎么通信的?

在一个集群里面, 不同节点需要互相通信, 交换信息. 这样才能知道, 哪些节点能正常通信, 哪些节点挂了, 哪些节点新加进来了.

redis集群是通过Gossip协议来做通信的.

- Gossip协议的思想就是说, 每个节点周期性地选择一些节点，将当前节点存的信息传播给这些节点.
- 收到信息的节点, 也干同样的事情, 也选一些节点, 然后传播信息. 这样很快所有节点的信息就都一致了.
- 这个过程就像谣言传播, 或者流行病, 瘟疫传播一样, 所以Gossip协议也叫谣言传播协议, 或者叫流行病传播协议.

redis集群利用Gossip协议来发送下面几种消息来互相传递信息.

- meet消息：通知新节点加入。新节点加入集群时，由现有节点主动发送meet消息.
- ping消息：节点会向集群中其他节点周期性发送ping消息 (包含自身状态、已知节点列表、哈希槽分配信息)
- pong消息：收到 PING 或 MEET 消息后的响应信息.
- fail消息：当某个节点被多数节点标记为下线时发送, 其他节点接收到fail消息之后把对应节点标记为下线
- ...

集群某个节点故障了怎么办?

- 由于redis集群采用Gossip协议来互相通信, 所以集群就不需要使用哨兵机制来监控了.
- 当一个节点认为另一个节点挂了, 这时候就认为该节点是主观下线.
- 当超过半数以上的节点认为某个节点挂了, 这时候就标记为客观下线. 然后通过Gossip协议广播 `FAIL`消息触发故障转移
    - 如果挂的是从节点, 那对应的主节点, 就把这个节点从主从复制的列表中移除. 然后无事发生.

- 如果挂的是主节点, 那其他主节点就会投票, 选出一个从节点, 然后转从为主. 然后由新主节点接管hash槽, 最后广播消息通知其他节点.